

BÀI 1 - DOCKER LÀ GÌ VÀ VÌ SAO CẦN DOCKER?

Lộ trình Docker cơ bản - hướng đến Docker Compose và CRUD nhỏ

Mục tiêu bài học: Sau bài này, bạn cần hiểu Docker giải quyết vấn đề gì, Container là gì ở mức khái niệm, Docker có vai trò nào đối với developer và Docker khác Virtual Machine ở điểm cơ bản nào.

1. Vấn đề thực tế khi chạy project trên nhiều máy

Giả sử bạn làm một project PHP trên máy của mình với PHP 8.4, MySQL 8 và Apache. Project chạy tốt. Khi gửi source cho người khác, máy của họ có thể dùng PHP 8.1, MySQL 5.7 hoặc cấu hình Apache khác. Khi đó project có thể lỗi dù source code giống nhau.

Đây là tình huống rất phổ biến: "Máy tôi chạy được nhưng máy bạn không chạy".

Nguyên nhân không chỉ nằm ở code mà còn ở sự khác nhau về runtime, thư viện, dependency, phiên bản phần mềm và cấu hình môi trường.

2. Cách triển khai truyền thống

Nếu không dùng Docker, mỗi người có thể phải tự cài PHP, Apache, MySQL, extension và tự cấu hình môi trường. Điều này tốn thời gian và dễ tạo ra sự khác biệt giữa các máy.

Máy A: PHP 8.4 + MySQL 8 + Apache

Máy B: PHP 8.1 + MySQL 5.7 + Apache cấu hình khác

=> Hai môi trường khác nhau có thể dẫn đến kết quả chạy khác nhau.

3. Docker giải quyết vấn đề như thế nào?

Docker giúp đóng gói và chạy ứng dụng trong các môi trường cô lập gọi là Container. Nhờ đó, developer có thể mô tả rõ môi trường cần thiết để ứng dụng chạy và giảm sự khác biệt giữa các máy.

Ứng dụng + Runtime + Dependencies + Configuration -> Docker -> Container

4. Docker là gì?

Docker là một nền tảng giúp đóng gói, phân phối và chạy ứng dụng trong các môi trường cô lập gọi là Container.

Docker không phải ngôn ngữ lập trình và cũng không phải framework. Nó không thay thế PHP, .NET, Java hay Node.js. Docker nằm ở tầng triển khai và môi trường chạy.

5. Container là gì?

Ở mức nhập môn, có thể hiểu Container là một môi trường chạy ứng dụng được cô lập. Một container có thể chạy PHP + Apache, một container khác có thể chạy MySQL, một container khác có thể chạy Redis.

PHP Container

↓

Docker Network
↓
MySQL Container

6. Docker không thay thế ngôn ngữ hay framework

Docker không thay PHP hoặc .NET. Ví dụ, bạn vẫn viết API bằng ASP.NET Core hoặc website bằng PHP. Docker chỉ giúp tạo môi trường để ứng dụng đó được chạy nhất quán hơn.

ASP.NET Core API -> Docker Container -> Hệ điều hành
PHP Application -> Docker Container -> Hệ điều hành

7. Docker không chỉ dùng khi deploy

Docker có thể được dùng ngay khi phát triển. Ví dụ, thay vì cài MySQL trực tiếp lên Windows, bạn có thể chạy MySQL trong một container. Khi không cần nữa, có thể xóa container mà không phải gỡ toàn bộ phần mềm khỏi máy.

8. Lợi ích chính của Docker đối với developer

Lợi ích	Ý nghĩa
Môi trường nhất quán	Giảm khác biệt giữa máy cá nhân, máy đồng đội và server.
Setup nhanh	Có thể dựng môi trường bằng cấu hình Docker thay vì cài thủ công nhiều công cụ.
Cô lập ứng dụng	Nhiều project có thể dùng các phiên bản runtime khác nhau.
Dễ tạo/xóa môi trường	Có thể thử database hoặc service mới mà không cần cài trực tiếp lâu dài.
Thuận tiện khi triển khai	Môi trường phát triển và triển khai có thể giống nhau hơn.

9. Docker và Virtual Machine có giống nhau không?

Không hoàn toàn. Virtual Machine thường có một Guest OS hoàn chỉnh. Container thường chia sẻ kernel của hệ điều hành host và đóng gói ứng dụng cùng các dependency cần thiết, vì vậy thường nhẹ hơn và khởi động nhanh hơn VM.

Virtual Machine: Guest OS + Application + Dependencies
Container: Application + Dependencies

Lưu ý: Docker và Virtual Machine phục vụ các mục đích khác nhau và không phải lúc nào cũng thay thế lẫn nhau.

10. Ví dụ liên hệ với .NET

Sau này nếu bạn có một ASP.NET Core API dùng SQL Server, bạn có thể tổ chức thành hai container: một container chạy API và một container chạy SQL Server. Hai container giao tiếp qua Docker Network và dữ liệu database có thể được lưu bằng Docker Volume.

ASP.NET Core Container
↓
Docker Network
↓
SQL Server Container
↓

11. Chuỗi tư duy cần nhớ

Ứng dụng cần môi trường -> Các máy có môi trường khác nhau -> Có thể phát sinh lỗi -> Docker giúp chuẩn hóa môi trường chạy -> Ứng dụng chạy trong Container

PHẦN CÂU HỎI ÔN TẬP VÀ ĐÁP ÁN

Câu 1: Docker giải quyết vấn đề gì khi project chạy trên nhiều máy?

Đáp án: Docker giúp giảm vấn đề khác biệt môi trường giữa các máy. Thay vì mỗi người phải tự cài và cấu hình công cụ theo cách khác nhau, Docker giúp tạo môi trường chạy nhất quán hơn để project có thể chạy ổn định trên nhiều máy.

Câu 2: Container có thể hiểu đơn giản là gì?

Đáp án: Container có thể hiểu đơn giản là một môi trường chạy ứng dụng được cô lập, chứa các thành phần cần thiết để ứng dụng hoạt động.

Câu 3: Docker có phải ngôn ngữ lập trình không?

Đáp án: Không. Docker là nền tảng dùng để đóng gói, chạy và quản lý container.

Câu 4: Docker có thay thế PHP hoặc .NET không?

Đáp án: Không. PHP, .NET, Java, Node.js... vẫn là công nghệ dùng để xây dựng ứng dụng. Docker chỉ hỗ trợ môi trường chạy và triển khai ứng dụng.

Câu 5: Tại sao chạy database bằng Docker có thể tiện hơn cài trực tiếp lên máy?

Đáp án: Vì có thể dùng image có sẵn để tạo database container nhanh, cô lập, dễ xóa và tạo lại mà không cần cài database trực tiếp lên máy. Image là bản mẫu; database thực sự chạy trong container.

Câu 6: Docker và Virtual Machine có phải hoàn toàn giống nhau không?

Đáp án: Không. Cả hai đều có tính cô lập, nhưng Virtual Machine thường có Guest OS hoàn chỉnh, còn container thường nhẹ hơn và chia sẻ kernel với hệ điều hành host.

Ghi chú về câu trả lời của bạn

Bạn đã trả lời đúng ý chính ở hầu hết các câu. Riêng câu 5 cần diễn đạt chuẩn hơn: không phải chỉ vì database được 'đóng gói thành image'. Image là bản mẫu; Docker dùng image đó để tạo ra container, và chính container mới là nơi database chạy. Điểm tiện lợi nằm ở việc tạo nhanh, cô lập, dễ xóa/tạo lại và không cần cài trực tiếp database lên máy.

Tự đánh giá sau Bài 1

Nếu bạn có thể tự giải thích được 6 câu trên mà không xem lại tài liệu, bạn đã đủ nền tảng để chuyển sang Bài 2 - Docker Architecture: Docker Client, Docker Engine, Registry và Docker Hub.